

Quantization Basics

Smaller, Faster, Cheaper Models with Lower Precision

A practical guide for ML engineers deploying models to production



What Is Quantization?

Quantization converts high-precision model weights and activations to lower-precision formats:

- FP32 → INT8 or FP16 conversion
- Represents values using **fewer bits**
- Dramatically reduces model footprint
- Critical for resource-constrained environments



FP32 values mapped to INT8 values with minimal information loss

Why Quantize?



Inference Efficiency

Lower latency and higher throughput for production workloads. Critical for real-time applications requiring sub-millisecond response times.



Cost Savings

Reduce cloud compute costs by 2-4x for large-scale inference workloads through more efficient hardware utilization.



Edge Deployment

Run sophisticated models on mobile devices, IoT sensors, and edge hardware with strict memory constraints.



Energy Efficiency

Lower power consumption for battery-powered devices and reduced carbon footprint for data centers.

Types of Quantization

1

Post-Training Quantization (PTQ)

Applied after model training is complete. Simple drop-in replacement with minimal engineering effort.

- No retraining required
- Quick to implement, moderate accuracy loss

2

Quantization-Aware Training (QAT)

Simulates quantization effects during training to maintain accuracy.

- Fine-tune with quantization in the loop
- Higher engineering cost, better results

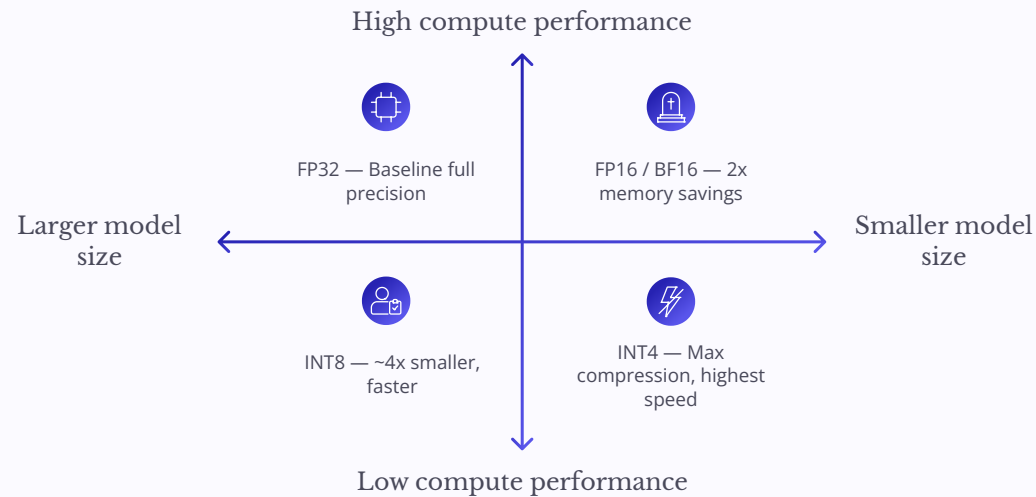
3

Dynamic vs. Static Quantization

Dynamic: Weights quantized offline, activations at runtime

Static: Both weights and activations quantized with calibration dataset

Precision Levels



FP32 & FP16/BF16

- **FP32:** Full precision baseline (32-bit)
- **FP16/BF16:** Half-precision formats
- Well-supported on modern GPUs
- 2x memory savings with minimal accuracy loss

INT8 & Beyond

- **INT8:** 8-bit integer quantization
- 4x smaller than FP32
- Widely supported in production hardware
- **INT4/INT2:** Extreme compression
- 8-16x smaller, emerging in specialized hardware

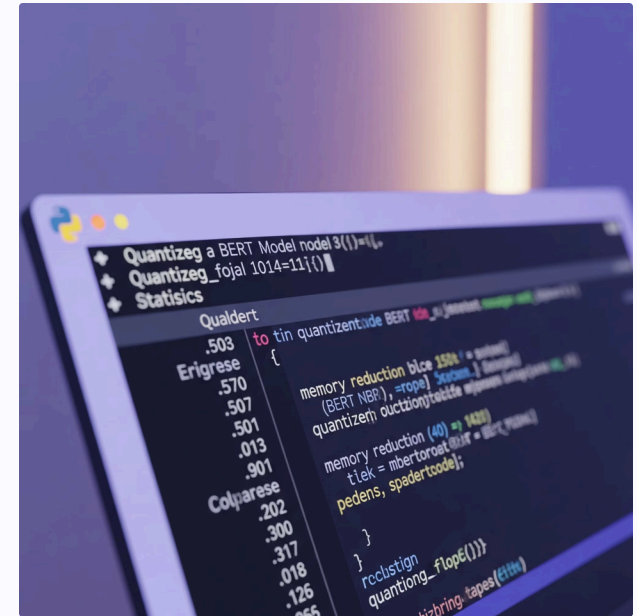
Example: PyTorch Dynamic Quantization

```
import torch
import torch.quantization
from transformers import BertModel

# Load full-precision model
model = BertModel.from_pretrained("bert-base-uncased")

# Apply dynamic quantization to Linear layers
quantized_model = torch.quantization.quantize_dynamic(
    model,
    {torch.nn.Linear},
    dtype=torch.qint8
)

# Check parameter reduction
print("Model size reduced:",
      model.num_parameters(), "→",
      quantized_model.num_parameters())
```



A 110M parameter BERT model can shrink from 420MB to around 105MB with INT8 quantization while maintaining 95-99% of original accuracy.

Benefits of Quantization

4x

Size Reduction

FP32 → INT8 conversion
reduces model size by
75%

3x

Throughput Gain

Typical performance
improvement for
inference workloads

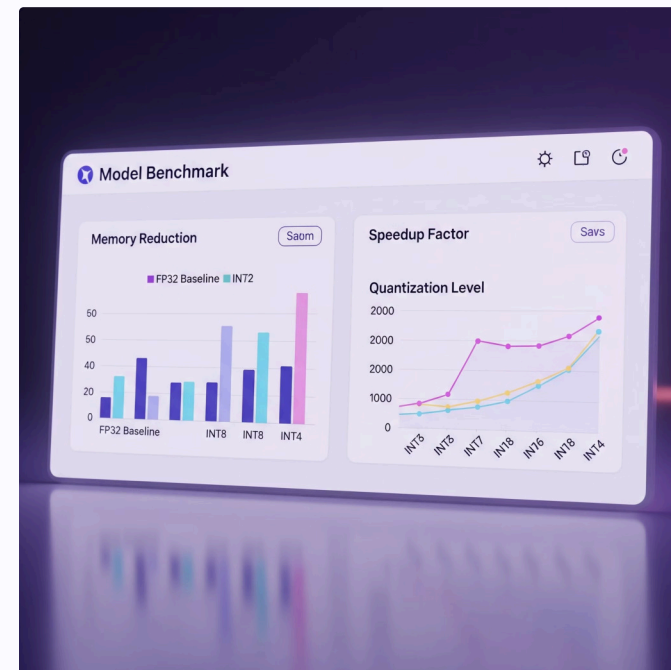
70%

Memory
Bandwidth

Reduction in memory
traffic for matrix
operations

Quantization can be stacked with other optimization techniques:

- Pruning (removing unnecessary weights)
- Knowledge distillation (teacher-student transfer)
- Compiler optimizations (TensorRT, ONNX Runtime)



Limitations of Quantization

Accuracy Impact

Some models experience significant degradation, particularly:

- NLP models with complex patterns
- Recommender systems with long-tail distributions
- Small models with limited redundancy

Hardware Compatibility

Not all hardware supports accelerated quantized operations:

- INT8: Widely supported (CPUs, modern GPUs)
- INT4/INT2: Limited to specialized hardware
- May need specific kernels or operator libraries

Engineering Complexity

Implementation challenges include:

- Calibration data collection and management
- QAT requires retraining infrastructure
- Model validation across precision levels

Where Quantization Shines

1 LLM Inference

Run larger language models within GPU memory constraints:

- Extend context window sizes
- Reduce GPU RAM requirements by 50-75%
- Enable efficient multi-tenant serving

2 Edge Deployment

Run sophisticated models on constrained devices:

- Smartphone applications
- IoT sensors and cameras
- Low-power edge devices

3 Cost-Efficient Batch Processing

Reduce inference costs for large-scale prediction tasks:

- Recommendation systems
- Content moderation
- Data labeling and enrichment



Real-world example: A 13B parameter LLM quantized to 4-bit precision can run on consumer GPUs with 24GB RAM instead of requiring 52GB+ for FP16.

Best Practices

Start Simple

Begin with dynamic INT8 quantization:

- Easiest to implement
- Minimal engineering overhead
- Establish baseline performance gains

Hardware-Specific Optimization

Test on your target deployment hardware:

- Different hardware accelerates different formats
- Use hardware-specific quantization libraries when available
- Benchmark latency, not just accuracy

Hybrid Approaches

Consider mixed precision and complementary techniques:

- Keep sensitive layers in higher precision
- Combine with knowledge distillation to recover accuracy
- Use QAT for models where accuracy is critical



Key Takeaways

Trade-Off Worth Making

Quantization offers substantial efficiency gains with manageable accuracy trade-offs.

FP32 → INT8 typically preserves >98% of model accuracy while delivering 3-4x performance benefits.

Implementation Options

Choose the right approach for your needs:

- PTQ: Quick wins, minimal engineering
- QAT: Maximum accuracy preservation

Production-Ready

INT8 quantization is mature and well-supported across frameworks:

- PyTorch, TensorFlow, ONNX Runtime
- Cloud and edge deployment

Quantization is no longer optional — it's a core part of the ML engineering toolkit for efficient model deployment at scale.